

In this lab assignment, you will explore various sampling-based methods for motion planning. You will analyze how different hyperparameters influence the convergence rate of these methods and affect the quality of the solutions obtained.

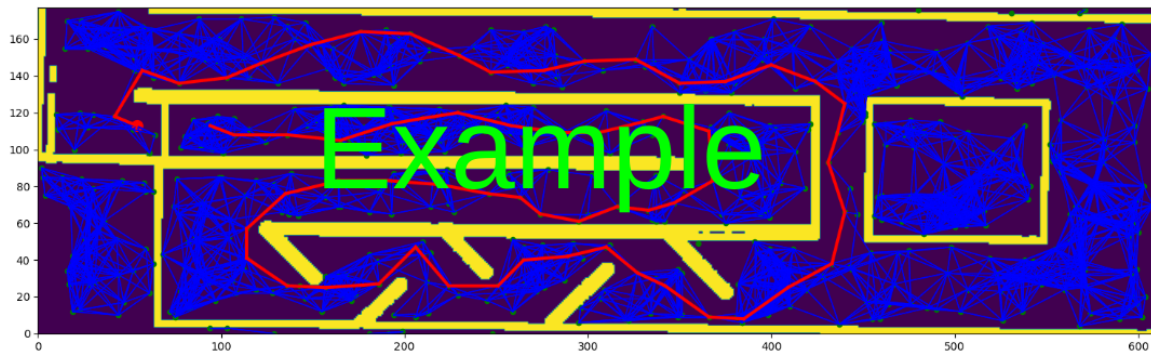
Part 1: PRM (50%)

You have been provided with a map named `"maze_test.npy"` stored as a numpy array. A value of "0" denotes a free cell where the robot can move, while all other values represent obstacles. Initially, we expand the original map using the `inflate()` function to ensure paths are sufficiently distanced from obstacles. Subsequently, we utilize the `CSpace` class to convert from meters to pixels. The parameter "resolution" within the `main()` function corresponds to 0.05 meters, approximately equivalent to 1 pixel. Please refrain from altering the parameters defined in the main function.

- Implement the `build_prm_graph()` function. This function constructs a graph from `self.map`, sampling `self.max_itr` vertices. Store the graph as `self.graph`, a dictionary where each key is a vertex and the corresponding value is a list of neighbors along with their costs (stored as tuples). Use the `find_neighbors_in_range()` method, which uses `self.max_dist` (in pixel units) to determine which vertices should be connected. Return a list of tuples ``[(neighbor, cost), ...]``. To calculate the cost, utilize the `get_cost()` method, using Euclidean distance.
- To determine if two neighbors can be connected, implement `local_planner()` as described in the lecture. This method verifies if a transition between two configurations is valid and should return ``True`` if valid.
- Implement the `is_in_collision()` method to determine if a given configuration is obstructed by obstacles. Return ``False`` if obstacle-free.
- Once the graph is constructed, initially add start and goal vertices to the graph. Implement the `A_Star` class to find the shortest path, with `h()` implementing the heuristic function and `find_path()` implementing the main A* algorithm. `reconstruct_path()` should return the vertices of the path.
- You may restructure the code as necessary but do not alter the provided parameters. Ensure the code runs correctly, as non-functional code will receive a grade of 0.

Assignments:

1. Execute the code with `max_itr` values of [700, 1000, 1500, 2000] and `dist= 30`. **Plot** the cost as a function of `max_itr`. **Plot** the running time as a function of `max_itr`. Provide an explanation of your results.
2. Include screenshots of the graph and path for each `max_itr` value using the `Plotter` class.



3. Convert the path with the lowest cost to the meter using the `Cspace.pathindex2pathmeter()` method and save it as a numpy array. Run the pure pursuit controller from HW1 to follow it and report the tracking error. Additionally, supply the tracking figure.

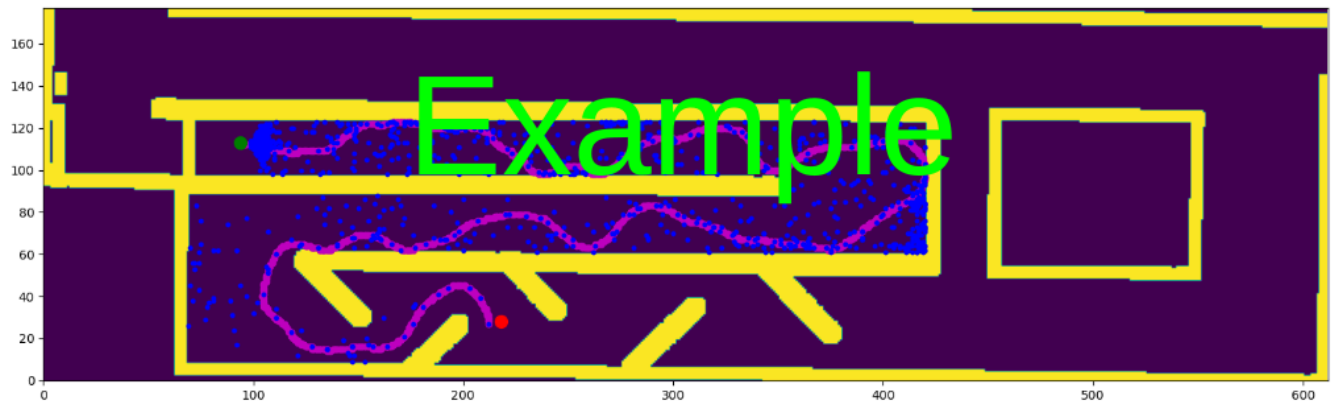
Part 2: Kino RRT (50%)

You have been provided with a map named "maze_test.npy" stored as a numpy array. A value of "0" denotes a free cell where the robot can move, while all other values represent obstacles. Initially, we expand the original map using the `inflate()` function to ensure paths are sufficiently distanced from obstacles. Subsequently, we utilize the `Cspace` class to convert from meters to pixels. The parameter "resolution" within the `main()` function corresponds to 0.05 meters, approximately equivalent to 1 pixel. Please refrain from altering the parameters defined in the `main` function.

- The `Odom` class implements the kinematics of Ackermann steering. Implement the `sample_control_command()` method in the `Odom` class, which returns time, steering, and velocity. These parameters are initialized in its `__init__()` function. The `propagate()` method is already implemented for your convenience.
 - You are provided with the implementation of the Tree data structure in `Py_Utils.py`. Familiarize yourself with the methods of the Tree class as they are trivial.
 - Implement the KINORRT algorithm as taught in the lecture. The `find_path()` method implements the main algorithm.
 - `sample()` should return a random configuration.
 - `local_planner()` should return `True` if the transition between two configurations is valid.
 - Use `self.tree.vertices[x_new_idx].set_waypoints(edge)` to store the waypoints of a given edge.
 - `is_in_collision()` should return `False` if the given configuration is obstacle-free.
 - `get_shortest_path()` returns the path and path indexes and cost
 - You have the flexibility to change the code structure as needed, but do not alter the provided parameters. Ensure the code runs correctly, as non-functional code will receive a grade of 0.
1. Run your code 10 times with `max_itr = 10000`, then **report** the statistics of the algorithm in a table format as shown in the example below. Additionally, **provide an explanation** of your results.

run	Iteration the path found	cost
1	3056	19785
2	Not found	-
10		

- Provide **plotting** of the vertices of the graph and the path found. Use the `Plotter()` class.



- Convert the path with the lowest cost to the meter using the `Cspace.pathindex2pathmeter()` method and save it as a numpy array. Run the pure pursuit controller from HW1 to follow it and report the tracking error. Additionally, supply the tracking figure.

